

INTERNAL BRIEFING · NOT AN OFFICIAL PALGATE DOCUMENT

GateAuto

A study guide to how the app links as a PalGate Linked Device, arms OS geofences and car Bluetooth, then opens a neighborhood gate — only after every safety check passes.

Package

com.gateauto.app

Stack

**Expo 57 · React Native 0.86 ·
TypeScript**

Codebase

C:\dev\GateAuto

Target

**Android first (Galaxy S25 Ultra) ·
same TS for iOS**

Contents

1. [01](#) What this app is, and what it is not
2. [02](#) The mental model — two switches, five triggers, one pipeline
3. [03](#) Architecture map and folder layout
4. [04](#) Boot sequence — how the process wakes
5. [05](#) Screens and navigation
6. [06](#) Linked Device auth
7. [07](#) PalGate API and the derived token
8. [08](#) Gate data model and storage
9. [09](#) The auto-open pipeline (read this twice)
10. [10](#) Proximity refine — fail closed
11. [11](#) Bluetooth, cooldown, safety lock
12. [12](#) Background survival
13. [13](#) UI layer
14. [14](#) Reading the log
15. [15](#) Tests, invariants, recommended study order

1. What this app is, and what it is not

GateAuto is a custom Expo Development Client. It pretends to be another PalGate “Linked Device” on your account, then opens gates over HTTPS when you actually arrive — not when GPS merely thinks you might have.

PalGate’s own AutoOpen is sometimes missing or imprecise. This app’s job is: **link once, pin each gate, arm monitoring, then open only when several independent checks agree.** There is no GateAuto cloud. Credentials never leave the phone except when you deliberately Export them. Opens go only to `https://api1.pal-es.com/v1/bt/` .

Not Expo Go

Classic Bluetooth and reliable Always-location / background geofencing need native modules. You build a custom client (`npx expo prebuild / expo run:android`) or an EAS APK. The `android/` and `ios/` folders are generated and gitignored.

Primary phone: Galaxy S25 Ultra. Same TypeScript runs on iOS, but car Bluetooth matching there is best-effort by accessory name, not Android classic BT.

2. The mental model

Almost every bug report and almost every “why didn’t it open?” log line collapses to this picture:

Master switch — Settings → Auto-open

Stored as `gateauto.monitoringEnabled` .
When off, OS geofences, BT connect watch, and the quiet foreground service are all torn down. Per-gate Auto toggles are grayed out. Nothing auto-opens.

Per-gate switch — Auto on each row

`GateConfig.enabled` . A gate also needs a map pin and radius before it is “geofence-ready.” Manual *Open* always works regardless of both switches.

When both are on, any of **five triggers** can start the same open pipeline. Triggers are how the phone *wakes up*. The pipeline is how it *decides*.

1 ENTER	2 EXIT	3 BT connect	4 Eligible-now	5 ~30s poll
---------	--------	--------------	----------------	-------------

Why five, not one

OS geofence ENTER only fires on a true boundary cross. If you live inside the circle (house inside the pin), ENTER never fires again until you leave and come back. That is why Bluetooth-connect, “eligible now” on app open, EXIT toward the street, and a quiet 30-second poll exist. Study `src/geo/geofencing.ts` header comments — they are the spec.

3. Architecture map

```
index.ts → App.tsx
├─ import src/geo/task.ts      // defineTask MUST run before JS is killed
├─ startMonitoringResyncLifecycle()
├─ ThemeProvider + RootNavigator
└─ GlobalOpenResultHost      // auto-open toast anywhere

UI  src/ui/screens + components
   ↓ calls
Bridge  src/integrations/optionalNative.ts  // soft-require geo + BT
   ↓
Geo      src/geo/geofencing.ts      ← heart of auto-open
        src/geo/refine.ts + proximity.ts
        src/geo/monitoringKeepAlive.ts
BT       src/bluetooth/*
API      src/palgate/api.ts + token.ts + linking.ts
Data     src/data/* (AsyncStorage + SecureStore)
```

Folder meaning

Path	Job
src/palgate/	Talk to PalGate: QR link, derived X-Bt-Token , GET devices, GET open-gate.
src/data/	On-device state: credentials, gates, event log, safety lock, cooldown helpers.
src/geo/	OS geofences, GPS refine, keep-alive FGS, boot resync, TaskManager handlers.
src/bluetooth/	Car head-unit connect listener + OR matching of saved devices.
src/ui/	Screens, theme (teal from the app icon), sheets, gate row, map.
src/notifications/	Quiet “GateAuto auto-open” channel + open/fail alerts.
src/integrations/	UI must not crash if native geo/BT failed to load; this is the soft bridge.
src/platform/	Expo config plugins (boot receiver, battery links).
tests/	Pure logic: haversine, refine rules, cooldown, burst lock, token, reorder. No device needed.

There is no Redux, no server, no GraphQL. State is AsyncStorage JSON plus SecureStore for the three credential fields. Headless Android tasks re-enter the same TypeScript modules.

4. Boot sequence

Headless geofence callbacks can run when the UI is not mounted. Expo TaskManager requires `defineTask` to have already executed in that JS runtime. That is why `App.tsx` starts with a side-effect import:

```
import './src/geo/task';
import { startMonitoringResyncLifecycle } from './src/geo/monitoringResync';
startMonitoringResyncLifecycle();
```

`src/geo/task.ts` registers:

- `GATEAUTO_GEOFENCE_TASK` — OS ENTER / EXIT → `handleGeofenceEnter` / `handleGeofenceExit`
- `GATEAUTO_MONITORING_KEEPALIVE_TASK` — ~30s FGS wake → `runEligibilityPoll`
- `GATEAUTO_LINKING_WATCH_TASK` — FGS only while waiting for QR scan (keeps long-poll alive)
- Legacy location-watch task — handler kept so old APKs don't crash; never started
- `GateAutoBootSync` — `AppRegistry` headless task after reboot → `syncGeofences()`

`startMonitoringResyncLifecycle` also runs `resyncMonitoringIfArmed('app-start')` and on AppState foreground. If the monitoring flag is on, it re-registers geofences (Android forgets them more often than you want) and runs eligible-now. Safe to call repeatedly; `syncGeofences` is idempotent.

5. Screens and navigation

src/navigation/RootNavigator.tsx

On launch: `hasCredentials()` → `GatesList` if linked, else `LinkAccount` . Stack screens:

Route	Title in UI	What to study
LinkAccount	Link PalGate	QR + long-poll; Import JSON
ExportAccount	Export	QR of credentials for phone-to-phone copy
Permissions	Permissions	Always location, notifications, BT, Unrestricted battery
GatesList	Gates	Pull devices, drag reorder, Open, per-gate Auto
GateEditor	Gate	Pin, radius, BT list, cooldown, Test Open, autosave
Monitoring	Log	Event list + safety-lock banner (not the master switch)
Settings	Settings	Master Auto-open, appearance, permissions, export, unlink

Heavy screens (editor, settings, export) use `getComponent={() => require(...)}` so the first paint of Gates does not load maps/native modules up front.

6. Linked Device auth

src/palgate/linking.ts · src/ui/screens/LinkAccountScreen.tsx · src/data/credentials.ts

1. `createLinkSession()` makes a UUID and QR payload exactly `{"id": "<uuid>"}` (space after the colon — PalGate’s scanner is picky; this matches pylgate / Homebridge).
2. Official PalGate app → Linked Devices → scan.
3. GateAuto long-polls `GET un/secondary/init/{uuid}`. PalGate holds the GET open until scan. The client reconnects about every 55s so a dead mobile connection does not sit forever.
4. On success, parse `user.id` (phone), `user.token` (hex session), `secondary` (token type, almost always SECONDARY = 2).
5. Save in SecureStore: `sessionToken`, `phoneNumber`, `tokenType`. Never in the event log.

Same-phone linking is fragile

Switching to PalGate to scan can kill the long-poll. The linking FGS + keep-awake + sticky “Keep GateAuto open” exist for that. Prefer split screen, or link on an emulator and Export → Import.

Unlink clears SecureStore and stops monitoring. Export is a personal transfer of the Linked Device session — anyone with that JSON can open your gates.

7. PalGate API and the derived token

`src/palgate/api.ts · token.ts · aes.ts`

Every authenticated call sends header `X-Bt-Token`, a 23-byte hex string derived from the session token, phone number, token type, and current unix time. Ported from `homebridge-palgate / pylgate`. Study `generateToken` then the tests in `tests/configAndSafety.test.ts` — they lock the bytes.

Important endpoints:

- `GET devices` — list PalGate devices; merged into local gates without scrambling user order.
- `GET device/{baseId}/open-gate?outputNum=N&_={cacheBust}` — the actual open. It is a GET (PalGate's design). OkHttp will happily reuse a cached 200, so the client sends `Cache-Control: no-store` and a unique `_` query. Without that, the log says success and the barrier never moves.

Multi-output devices (one PalGate box, several barriers) become ids like `abc123:2`. `splitDeviceId` peels off the output number. Device list parsing lives in `gatesStore.parseDevicesResponse` (Homebridge-compatible `outputN` / `nameN` fields).

User-Agent is spoofed as `okhttp/4.9.3` to look like the official Android app.

8. Gate data model

`src/data/gatesStore.ts`

Key `gateauto.gates` in AsyncStorage. One record per barrier:

Field	Meaning
<code>id</code> / <code>deviceId</code>	Stable PalGate id, possibly <code>base:outputNum</code>
<code>name</code> / <code>nameOverride</code>	PalGate name vs user rename
<code>enabled</code>	Per-gate Auto (ignored unless master monitoring is on)
<code>lat</code> , <code>lng</code> , <code>radiusMeters</code>	Pin. Default radius 50 m, slider 25–150 m
<code>cooldownMs</code>	Wait after a successful auto-open. Default 30 s. Values < 1000 are treated as <i>seconds</i>
<code>bluetooth</code>	<code>{ required, devices[] }</code> — OR list; address preferred over name
<code>lastOpenedAt</code> / <code>lastResult</code>	Cooldown clock + last outcome string

A gate is geofence-ready only when enabled + finite lat/lng + radius > 0 (`isGeofenceReady`). Refreshing PalGate devices `mergeDevicesIntoGates` keeps existing pins and the user's drag order (`src/data/gateOrder.ts`).

9. The auto-open pipeline

`src/geo/geofencing.ts` – start here after you understand auth

Every trigger funnels into the same guards, then `performOpen`. Manual Open on the list **skips** this pipeline (no refine, no BT, no safety lock, no cooldown).

Guard order (ENTER as the textbook path)

1. Gate exists and `enabled`
2. Pin/radius present
3. **ENTER-only**: ignore if geofences were re-registered in the last 60 s (already-inside spam when the OS fires ENTER on sync)
4. Safety lock — this gate not burst-locked
5. Cooldown — `now - lastOpenedAt >= gate.cooldownMs`
6. Proximity refine — fresh high-accuracy GPS, fail closed
7. Car Bluetooth if required — with a retry window on ENTER/EXIT (~3 s × ~45 s) because the head unit often connects after you are already in the circle
8. `performOpen` — load credentials, `openGate`, record success/fail, notify, publish UI result

The five triggers

Trigger	Function	When	Radius rule
ENTER	<code>handleGeofenceEnter</code>	True cross into the circle	$\leq$ radius, cap 250 m
EXIT	<code>handleGeofenceExit</code>	Leave toward the street (home fences)	$\leq$ radius $\times$ 1.1, still cap 250 m. No 60 s sync debounce
BT connect	<code>handleBluetoothDeviceConnected</code>	Car ACL/A2DP connects, monitoring on, device matches a gate's OR list	Same as ENTER. If you are not inside → log <code>bt_connect_outside</code> , no open
Eligible-now	<code>checkEligibleNowAndOpen</code>	Arm monitoring, app foreground, resync. Already inside + BT already connected produces no ENTER/ACL	Same as ENTER. Light 5 s check throttle; opens still gated only by cooldown
Poll	<code>runEligibilityPoll</code>	~30 s keep-alive wake while armed, app in background	Same. Min gap 25 s. Far-away skip logs throttled to 5 min

performOpen is the only auto path that talks to PalGate

It also calls `recordSuccessfulOpen` (burst lock), sets `lastOpenedAt` (cooldown), appends a typed log event, `publishOpenResult` (in-app flash), and a notification. Failures go to `*_error` kinds and never increment the burst lock.

10. Proximity refine — fail closed

`src/geo/refine.ts` · `src/geo/proximity.ts` · `src/geo/haversine.ts`

A geofence ENTER from the OS is not trusted. Before any auto-open, `refineArrival` takes up to 4 high-accuracy fixes. Each fix must be:

- Fresh — age ≤ 25 s (background wakes often return a slightly old sample; hours-old lastKnown is rejected)
- Near the pin — haversine distance $\leq$ radius (ENTER/BT/poll) or radius $\times 1.1$ (EXIT)
- Never farther than **250 m**, even if someone set a huge radius
- Accurate enough — worse than 60 m rejected; 50–60 m only if distance + accuracy still fits inside the fence

These helpers are **pure** (except the GPS call). That is why they are unit-tested without a phone. If you change a number here, change the test and this mental model together.

False EXIT while kilometers away

Cheap phones sometimes fire EXIT with a wild coordinate. Refine is mandatory on EXIT too. Far GPS → `skipped_refine`, barrier stays closed.

11. Bluetooth, cooldown, safety lock

Bluetooth

`src/bluetooth/match.ts` · `btConnectMonitor.ts` · `carBluetooth.ts` · `androidProfiles.ts`

If a gate's `bluetooth.required` is true, auto-open needs at least one listed device connected (OR). Empty list while required fails closed. Address match (normalized colons) beats name. Android listens for classic BT connect; iOS is weaker.

BT-connect is the *primary* home trigger: you start the car in the driveway (already inside the fence), the head unit connects, GateAuto opens. ENTER will not fire because you never left.

Cooldown

`src/geo/cooldown.ts` · `src/data/cooldownNormalize.ts`

After a successful auto-open, that gate will not auto-open again until `gate.cooldownMs` elapses. Default 30 seconds. There is no extra hidden 90 s debounce on opens. The 60 s window after geofence sync only suppresses ENTER, not EXIT or BT.

`normalizeCooldownMs` : stored values ≥ 1000 are milliseconds; values in (0, 1000) are seconds. A stored `10` is 10 s, not 10 minutes. That bug existed; the tests exist so it does not come back.

Safety lock (auto only)

`src/data/safetyBurst.ts` · `openSafetyLock.ts`

4 successful auto-opens of the same gate within 2 minutes → that gate locks for 40 minutes.

Manual Open still works. Purpose: a bad pin or GPS jitter must not hammer a neighbor's barrier. State is per-gate in AsyncStorage. `applyBurstOpen` is pure and unit-tested.

12. Background survival

`src/geo/monitoringKeepAlive.ts` · `monitoringResync.ts` · `src/platform/withAndroidBootSync`

Situation	Does auto-open still work?
App backgrounded or swiped away	Yes, if Unrestricted battery — quiet FGS + OS geofences + BT receiver
Phone reboot	Yes after boot re-registration, if monitoring was left ON
Force stop (App info)	No. Android kills FGS, geofences, BT, boot, alarms. Opening the app restores if the flag is still ON
OEM “deep sleeping”	May die until next open — keep Unrestricted

The keep-alive is *not* continuous high-accuracy GPS. It is a silent min-importance notification plus Balanced location updates with a 30 s time interval so Samsung/Doze actually wakes the JS to poll. Each wake takes one high-accuracy shot inside `runEligibilityPoll`.

Phone setup that is part of the architecture: Location **Always**, notifications allowed, Bluetooth allowed, battery Unrestricted. “While using” location means geofences will not fire in the background. That is an OS rule, not an app bug.

13. UI layer

`src/ui/theme.ts · screens/* · components/*`

Visual identity is the app icon: light mist + deep teal `#1A4A47`, dark cabin `#071314` / pine `#3D9A78`. `ThemeProvider` maps that onto React Navigation.

Worth knowing while you read screens:

- `GatesListScreen` — drag-reorder via a left notch (`PanResponder`). Pull-to-refresh is disabled while dragging. Per-gate Auto is locked when Settings Auto-open is off; tapping it points you at Settings.
- `GateEditorScreen` — autosave, no Save button. First pin is silent; changing an existing pin asks via `ConfirmSheet`. GPS uses Balanced + last-known, not High, so “Use current location” does not hang.
- `AutoOpenSettings` — master switch. Turns monitoring on without waiting for the GPS eligible-now pass (that runs in the background). Android Switches that are `disabled` while toggling often keep the old thumb — so this switch is not disabled during the call.
- `BarrierMark` — logo vs watching (radio arcs, arm down) vs opening (arm rises only for a real Open).
- `optionalNative.ts` — UI calls `tryStartGeofencing` / `tryStopGeofencing` so a missing native module degrades instead of crashing.

14. Reading the log

src/data/eventLog.ts · eventFilters.ts · MonitoringScreen.tsx

Last 100 events, newest first, AsyncStorage. The Log screen hides `test_*` kinds (Gate Editor “Test Open”). When something fails in the field, the `kind` is the diagnosis:

kind	Meaning
<code>opened</code> / <code>exit_open</code> / <code>bt_connect_open</code> / <code>eligible_now_open</code> / <code>poll_open</code>	API said ok, along that trigger
<code>skipped_refine</code>	GPS too far, too old, or too inaccurate — correct fail-closed
<code>skipped_bt</code> / <code>exit_skipped_bt</code>	Car not connected after the retry window
<code>bt_connect_outside</code>	Heard the car, but you were not at the pin
<code>cooldown</code>	Too soon after last successful auto-open
<code>safety_lock</code>	4 autos in 2 min; wait ~40 min; manual Open still works
<code>monitoring_armed</code>	Master switch turned on and geofences registered
<code>error</code>	Permissions, API, or arming failure — read the message

Messages often include distance and accuracy in meters. Tune radius and cooldown from those numbers, not from guesswork.

15. Tests, invariants, study order

`npm test` · `npm run typecheck`

`tests/autoOpenLogic.test.ts` — distance, EXIT factor, 250 m cap, accuracy ball, cooldown, BT OR matching, reorder.

`tests/configAndSafety.test.ts` — cooldown normalization, Bluetooth migration, display names, log filters, 4/2 min → 40 min lock, token bytes, deviceId split.

Invariants to memorize before changing auto-open

Far-away cap is 250 m. EXIT factor is 1.1. Cooldown is per-gate `cooldownMs` (sub-1000 = seconds). Safety lock is auto-only, 4 opens / 2 min → 40 min. Bluetooth is OR. open-gate GET must cache-bust. After Force Stop, user must open the app. Do not reintroduce continuous high-accuracy GPS as the keep-alive.

Recommended reading order

1. `README.md` — product contract and phone setup (Always + Unrestricted).
2. `App.tsx` + `src/geo/task.ts` — process model.
3. `src/palgate/linking.ts` then `token.ts` then `api.ts` — how a Linked Device becomes an open command.
4. `src/data/gatesStore.ts` — the record the rest of the app orbits.
5. `src/geo/geofencing.ts` from the file header through `handleGeofenceEnter`, `handleBluetoothDeviceConnected`, `performOpen`. This is the course.
6. `src/geo/proximity.ts` + `refine.ts` — why it refuses to open.
7. `src/data/safetyBurst.ts` + `cooldown.ts` — why it refuses to open *again*.
8. `src/bluetooth/match.ts` + `btConnectMonitor.ts` — the driveway case.
9. UI last: `GatesListScreen`, `GateEditorScreen`, `AutoOpenSettings`. They are consumers of the modules above.

GateAuto is not an official PalGate product. This briefing describes the codebase as of the study-guide export. When code and this PDF disagree, trust the TypeScript and the tests.